

Verslag Fase 3 Programmeerproject 2

Naam: Isler Gabriel

Studentennummer: 0631137

E-mailadres: gabriel.isler@vub.be

Studierichting: Computerwetenschappen

Academiejaar: 2025-2026

Inhoudsopgave

1. Inleiding.....	2
2. Overzicht van de functionaliteiten	2
3. Beschrijving van de ADT's.....	3
3.1 Trein ADT	3
3.2 Slagboom ADT	4
3.3 Verkeerslicht ADT	4
3.4 Wissel ADT	5
3.5 Spoor netwerk ADT	5
3.6 Reservatie ADT.....	6
3.7 Infrabel ADT	6
3.8 NMBS ADT	8
3.9 Railway-NMBS ADT	8
3.10 Execution Facade.....	8
3.11 Scenario	9
3.12 Verduidelijking	9
4. Afhankelijkheidsdiagram.....	10
.....	10
.....	10

1. Inleiding

Programmeerproject 2 is een project dat als doel heeft een softwaresysteem te creëren voor de besturing van een modelspoor. De software dient in staat te zijn om treinen te laten rijden, wissels en seinen te bedienen, en de veiligheid op het spoor te garanderen door middel van botsingspreventie. Al dit op een geautomatiseerde manier.

Om de ingewikkeldheid van het project te beheersen, wordt het systeem verdeeld in twee belangrijke onderdelen, waarbij elk een duidelijke verantwoordelijkheid heeft:

- Infrabel (server) is de essentie van het systeem en draagt de verantwoordelijkheid voor de spoorlogica en infrastructuur. Deze component zorgt ervoor dat de spoorblokken, wissels, treinen en seinen in goede staat zijn. Nieuw in deze fase is dat Infrabel volledig geautomatiseerde routes kan uitvoeren: gegeven een startblok en een bestemming, berekent het systeem zelf het pad door het spoornetwerk, reserveert de nodige infrastructuur, en stuurt de trein veilig naar het doel. Infrabel richt zich ook op alle veiligheidsaspecten, waaronder het voorkomen van botsingen via een reservatiesysteem en actieve monitoring van de bezette blokken.
- NMBS (GUI / Client) dient als een gebruikersinterface en maakt gebruik van een TCP-verbinding om op afstand met Infrabel te communiceren met netwerkberichten (s-expressies). De GUI laat de huidige status van het spoornetwerk zien en biedt de gebruiker de mogelijkheid om taken uit te voeren, zoals het starten van treinen, het aanpassen van wisselstanden, of het opgeven van een bestemming voor een trein. Aan de client-kant houdt een afzonderlijk Railway-NMBS-ADT een actuele cache bij van wissels, slagbomen, lichten en treinen, zodat de GUI snel en zonder netwerk-roundtrip kan teruggeven wat ze moet tonen.

De NMBS heeft geen spoorlogica. Zij maakt enkel gebruik van de informatie die door Infrabel wordt verzonden. Deze opsplitsing weerspiegelt de rolverdeling in een realistische spooromgeving.

Deze architectuur verhoogt de modulariteit en onderhoudbaarheid. Voor de graaf-functionaliteit is gebruikgemaakt van de a-d-bibliotheek uit het cursusmateriaal.

2. Overzicht van de functionaliteiten

Fase 3 breidt het hardware-onafhankelijke client-server systeem uit fase 2 uit met geautomatiseerde trajectplanning en concurrente veiligheid. De belangrijkste vernieuwingen en functionaliteiten zijn:

- Geautomatiseerde routing: De gebruiker geeft een bestemming op voor een trein, waarna Infrabel zelf het kortste pad berekent (Breadth-First Traversal over een graaf-model van het spoor) en de trein hop voor hop naar het doel stuurt.

Wissels worden automatisch op de juiste stand gezet en de rijrichting wordt indien nodig omgekeerd via een keer-detectie.

- Reservatiesysteem: Voor elke hop reserveert de trein atomair de wissels en het bestemmingsblok die hij nodig heeft. Twee treinen kunnen nooit dezelfde infrastructuur claimen, waardoor botsingen op gedeelde sporen onmogelijk zijn. Een trein die geblokkeerd wordt door een reservatie wacht automatisch tot de infrastructuur vrijkomt.
- Botsing-detectie en noodstop: Een achtergrond-thread monitort continu de bezette detectieblokken. Bij twee treinen op hetzelfde blok, of bij een manueel bestuurde trein die richting een bezet blok rijdt, volgt een automatische noodstop die alle treinen onmiddellijk stilzet.
- Validatie op fysieke hardware: Het systeem is uitvoerig getest op zowel de virtuele simulator als de Z21-hardware-baan, waarbij dezelfde commando's beide modi correct aansturen via de Execution Facade uit fase 2.

3. Beschrijving van de ADT's

3.1 Trein ADT

Beschrijving: Het ADT Trein modelleert een locomotief met identiteit, snelheid en positie. Maak-trein creëert een trein en registreert deze op de hardware; set-snelheid!, stop-trein! en verwijder-trein! sturen hem aan; geef-positie geeft het paar (vorig, huidig). Nieuw in deze fase: een trein houdt een bestemming bij (set-bestemming!, geef-bestemming), zijn positie wordt actief bijgewerkt door Infrabel via set-huidig!, en hij bewaart zijn actieve route-thread zodat Infrabel een lopende route kan stoppen of vervangen.

Operaties:

NAAM	SIGNATUUR
<i>maak-trein</i>	(t-id, huidig-segment, vorig-segment, hardware, vorig-voor-sim → trein)
<i>set-snelheid!</i>	(number → Ø)
<i>stop-trein!</i>	(Ø → Ø)
<i>is-op-spoor?</i>	(Ø → boolean)
<i>geef-snelheid</i>	(Ø → number)
<i>geef-id</i>	(Ø → number)
<i>verwijder-trein!</i>	(Ø → Ø)
	(Ø → list)

<i>geef-positie</i>	$(\emptyset \rightarrow \text{symbol})$
<i>geef-huidig</i>	$(\emptyset \rightarrow \text{symbol})$
<i>geef-vorig</i>	$(\emptyset \rightarrow \text{symbol})$
<i>geef-bestemming</i>	$(\text{symbol} \rightarrow \emptyset)$
<i>set-bestemming!</i>	$(\emptyset \rightarrow \text{thread v \#f})$
<i>geef-route-thread</i>	$(\text{thread v \#f} \rightarrow \emptyset)$
<i>set-route-thread!</i>	

3.2 Slagboom ADT

Beschrijving: Het ADT Slagboom modelleert een spoorwegoverweg-slagboom met twee toestanden: open of gesloten. Maak-slagboom creëert een slagboom met een uniek id; open-slagboom! en sluit-slagboom! veranderen de stand; geef-status geeft de huidige toestand terug en geef-id het id. De slagboom communiceert via de Execution Facade met de simulator of hardware.

Operaties:

NAAM	SIGNATUUR
<i>maak-slagboom</i>	$(\text{id, hardware} \rightarrow \text{slagboom})$
<i>open-slagboom!</i>	$(\emptyset \rightarrow \emptyset)$
<i>sluit-slagboom!</i>	$(\emptyset \rightarrow \emptyset)$
<i>geef-id</i>	$(\emptyset \rightarrow \text{symbol})$
<i>geef-status</i>	$(\emptyset \rightarrow \text{symbol})$

3.3 Verkeerslicht ADT

Beschrijving: Het ADT Verkeerslicht modelleert een spoorsein dat het treinverkeer regelt. Het houdt de huidige lichtstand bij (bv. Hp0 rood, Hp1 groen) en kan via veranderlicht! een nieuwe code krijgen. Maak-verkeerslicht creëert een sein met een uniek id; geef-kleur geeft de actuele kleur terug. Initieel staat het sein op rood. De aansturing verloopt via de Execution Facade.

Operaties:

NAAM	SIGNATUUR
<i>maak-verkeerslicht</i>	$(\text{id, hardware} \rightarrow \text{verkeerslicht})$
<i>geef-id</i>	$(\emptyset \rightarrow \text{symbol})$

<i>geef-kleur</i>	$(\emptyset \rightarrow \text{symbol})$
<i>verander-licht!</i>	$(\text{symbol} \rightarrow \emptyset)$

3.4 Wissel ADT

Beschrijving: Het ADT Wissel modelleert een spoorwissel met een uniek id en een stand die bepaalt welk pad de trein neemt. Maak-wissel creëert een wissel; set-stand! wijzigt de stand naar een nieuw nummer; geef-wissel-stand geeft de actuele stand terug. De aansturing verloopt via de Execution Facade, die complexe gevallen zoals de driewegwissel S-2-3 onder de motorkap correct vertaalt.

Operaties:

NAAM	SIGNATUUR
<i>maak-wissel</i>	$(\text{id, hardware} \rightarrow \text{wissel})$
<i>geef-id</i>	$(\emptyset \rightarrow \text{symbol})$
<i>geef-wissel-stand</i>	$(\text{symbol} \rightarrow \text{number})$
<i>set-stand!</i>	$(\text{number} \rightarrow \emptyset)$

3.5 Spoor netwerk ADT

Beschrijving: Het ADT Spoor netwerk modelleert de baan als een gerichte graaf waarin elk detectieblok (1-1, 1-2, ..., 2-8) een knoop is en elke fysieke verbinding een edge. Op elke edge staat als label de lijst wissel-instellingen ((wissel-id . stand) ...) die ingesteld moeten zijn om die verbinding te kunnen gebruiken. Dit ADT vormt de structuur waarop routes worden gezocht en wissels automatisch ingesteld. Maak-spoor netwerk bouwt de graaf eenmalig op met alle DB's en hun verbindingen. Geef-alle-dbs geeft de lijst van alle detectieblokken; adjacent? test of er een directe verbinding bestaat tussen twee DB's; geef-wissel-instellingen geeft voor een verbinding terug welke wissels op welke stand moeten staan; geef-buren geeft per DB alle directe burenen met hun bijhorende wissels. Onder de motorkap maakt deze ADT gebruik van de a-d-bibliotheek (adjacency-matrix en bft-labeled).

Operaties:

NAAM	SIGNATUUR
<i>maak-spoor netwerk</i>	$(\emptyset \rightarrow \text{spoor netwerk})$
<i>geef-alle-dbs</i>	$(\emptyset \rightarrow \text{list})$
<i>adjacent?</i>	$(\text{symbol, symbol} \rightarrow \text{boolean})$

geef-wissel-instellingen	(symbol, symbol $\rightarrow$ list-of-pairs v #f)
geef-buren	(symbol $\rightarrow$ list-of-pairs)

3.6 Reservatie ADT

Beschrijving: Het ADT Reservatiesysteem beheert wie welk infrastructuurelement (wissel of detectieblok) tijdelijk in gebruik heeft. Wanneer een trein een hop wil maken, claimt hij eerst atomair alle benodigde elementen via reserveer!. Als minstens één element al bezet is door een andere trein, faalt de claim volledig (geen partiële reservatie). Dit garandeert dat twee treinen nooit dezelfde infrastructuur tegelijk kunnen gebruiken, en vormt zo de kern van de botsingspreventie. Het ADT is intern thread-safe: elke operatie wordt beschermd door een semaphore, waardoor parallelle route-threads veilig kunnen lezen en schrijven. Geef-vrij! en verwijder-trein! maken elementen weer beschikbaar; wis-alles! wist alles in één klap (gebruikt door noodstop); eigenaar geeft terug welke trein een element claimt; voer-uit-als-vrij! combineert atomair een check met een actie en wordt gebruikt om manuele wissel-bediening te weigeren wanneer de wissel gereserveerd is.

Operaties:

NAAM	SIGNATUUR
maak-reservatiesysteem	($\emptyset \rightarrow$ reservatiesysteem)
reserveer!	(t-id, list $\rightarrow$ boolean)
verwijder-trein!	(t-id $\rightarrow \emptyset$)
wis-alles!	(element $\rightarrow$ t-id v #f)
eigenaar	(element $\rightarrow$ t-id v #f)
voer-uit-als-vrij!	(element, thunk $\rightarrow$ #f v t-id)

3.7 Infrabel ADT

Beschrijving: Het Infrabel-ADT is de server-zijde kern die alle spoorlogica en infrastructuur coördineert. Bij maak-infrabel wordt een modus ('SIM of 'HARDWARE) meegegeven die de juiste Execution Facade activeert, waardoor Infrabel volledig hardware-onafhankelijk blijft. Initialiseer-spoornetwerk! opent de verbinding en installeert alle statische componenten (wissels, slagbomen, seinen). Daarna brengt Infrabel de hoofd-ADT's (Trein, Wissel, Slagboom, Verkeerslicht) samen met het Spoor netwerk en het Reservatiesysteem onder één coördinerende laag.

Nieuw in fase 3 is dat Infrabel geautomatiseerde routes uitvoert: bij set-trein-bestemming! zoekt Infrabel zelf het pad via een BFT-algoritme, start een route-thread per

trein, reserveert per hop atomair de nodige wissels en blokken, en stuurt de trein aan tot hij arriveert. Een bezette-blokken-poller draait continu op de achtergrond om treinposities bij te werken en twee veiligheidschecks uit te voeren: botsing-detectie wanneer twee treinen hetzelfde blok delen, en een proactieve check die manueel rijdende treinen stopt zodra een buur-blok bezet is. Noodstop! zet alle treinen onmiddellijk stil, beëindigt hun route-threads en wist alle reservaties. Manuele set-wissel-stand!-aanvragen worden geweigerd wanneer de wissel door een andere trein gereserveerd is.

Operaties:

NAAM	SIGNATUUR
<i>maak-infrabel</i>	(symbol $\rightarrow$ infrabel)
<i>initialiseer-spoornetwerk!</i>	($\emptyset \rightarrow \emptyset$)
<i>voeg-trein-toe!</i>	(symbol, symbol, symbol $\rightarrow \emptyset$)
<i>verwijder-trein!</i>	(symbol $\rightarrow \emptyset$)
<i>zet-trein-snelheid!</i>	(symbol, number $\rightarrow \emptyset$)
<i>geef-trein-snelheid</i>	(symbol $\rightarrow$ number)
<i>stop-trein!</i>	(symbol $\rightarrow \emptyset$)
<i>set-wissel-stand!</i>	(symbol, number $\rightarrow \emptyset$)
<i>open-slagboom!</i>	(symbol $\rightarrow \emptyset$)
<i>sluit-slagboom!</i>	(symbol $\rightarrow \emptyset$)
<i>geef-slagboom-status</i>	(symbol $\rightarrow$ symbol)
<i>verander-licht!</i>	(symbol, symbol $\rightarrow \emptyset$)
<i>geef-lichtkleur</i>	(symbol $\rightarrow$ symbol)
<i>geef-wissel-stand</i>	(symbol $\rightarrow$ number)
<i>geef-alle-wissel-standen</i>	($\emptyset \rightarrow$ list)
<i>geef-alle-trein-ids</i>	($\emptyset \rightarrow$ list)
<i>geef-slagboom-statussen</i>	($\emptyset \rightarrow$ list)
<i>geef-licht-kleuren</i>	($\emptyset \rightarrow$ list)
<i>geef-bezette-detectieblokken</i>	($\emptyset \rightarrow$ list)
<i>geef-trein-positie</i>	(symbol $\rightarrow$ list)

geef-trein-bestemming	(symbol $\rightarrow$ symbol)
set-trein-bestemming!	(symbol, symbol $\rightarrow \emptyset$)
noodstop!	($\emptyset \rightarrow \emptyset$)

3.8 NMBS ADT

Beschrijving: Het NMBS-ADT fungeert als netwerk-client en vormt de brug tussen de GUI en de Infrabel-server. Het bevat geen eigen spoorlogica en werkt puur als RPC-proxy: elke aanroep wordt verpakt als een Racket-S-expressie, via write + flush-output naar de server gestuurd, waarna het ADT met read blokkeert tot het antwoord binnenkomt. Dankzij deze abstractie kan de GUI Infrabel-operaties aanroepen alsof ze lokaal draaien, terwijl het zware werk op afstand gebeurt. In fase 3 ondersteunt NMBS naast de bestaande commando's ook de nieuwe routerings-operaties van Infrabel (zoals set-trein-bestemming!, noodstop! en geef-trein-bestemming). Aan de client-kant werkt NMBS samen met het Railway-NMBS-ADT, dat een actuele cache van de netwerktoestand bijhoudt.

Operaties:

NAAM

SIGNATUUR

maak-nmbs	(input-port, output-port $\rightarrow$ nmbs)
------------------	--

De publieke methoden die NMBS aanbiedt, vormen een exacte weerspiegeling van de Infrabel-operaties, maar vertalen deze enkel naar netwerkberichten.

3.9 Railway-NMBS ADT

Beschrijving: Het Railway-NMBS-ADT houdt aan de client-kant een schaduw-kopie bij van de toestand van het spoornetwerk: lijsten van wissels, slagbomen, lichten, treinen en bezette blokken. De GUI raadpleegt deze cache voor weergave, zodat tekenen vlot blijft zonder voor elk frame een netwerkverzoek naar de server te sturen.

De ADT biedt twee soorten operaties: lezers (zoals geef-alle-wissel-standen, geef-slagboom-statussen, geef-licht-kleuren, geef-alle-trein-ids, geef-bezette-blokken) die door de GUI gebruikt worden, en updaters (zoals update-wissel-stand!, update-slagboom-status!, update-lichtkleur!, update-trein-snelheid!, voeg-trein-toe!, verwijder-trein!) die het NMBS-ADT aanroept telkens als een statusbericht van de server binnenkomt. Noodstop! zet alle gecachte treinsnelheden naar nul, in lijn met wat de server doet. *(Volledige tabel weggelaten ter vermijding van redundantie.)*

3.10 Execution Facade

Beschrijving: Dit ADT implementeert het Facade-ontwerppatroon en verbergt de complexiteit van de twee onderliggende API's (de virtuele simulator en de fysieke Z21-hardware) voor de Infrabel-server. Bij maak-hardware-facade wordt een modus

meegegeven ('SIM of 'HARDWARE); een hogere-orde functie kies-in-functie-van-modus dispatcht daarna elke aanroep naar de juiste implementatie. Hierdoor kan Infrabel naadloos wisselen tussen simulator en hardware zonder een regel code te veranderen.

Operaties:

NAAM	SIGNATUUR
<i>maak-hardware-facade</i>	(symbol $\rightarrow$ execution-facade)
<i>setup!</i>	($\emptyset \rightarrow \emptyset$)
<i>start!</i>	($\emptyset \rightarrow \emptyset$)
<i>stop!</i>	($\emptyset \rightarrow \emptyset$)

Om herhaling in dit document te vermijden, zijn niet alle methoden van dit ADT expliciet opgelijst. De dispatcher van de Execution Facade bevat een reeks commando's (zoals set-wissel-stand!, registreer-trein!, open-slagboom!, etc.) wiens signatuur nagenoeg identiek is aan de methoden binnen het Infrabel-ADT.

3.11 Scenario

Beschrijving: Het Scenario-ADT maakt de netwerktoestand persistent: het slaat de volledige actuele opstelling op in een tekstbestand en kan een eerder opgeslagen toestand opnieuw inladen. Maak-scenario koppelt het ADT aan een nmbs-object, dat fungeert als bron voor alle statusinformatie. Bij sla-scenario-op! verzamelt het ADT alle relevante data van wissels, slagbomen, lichten en treinen (inclusief posities, snelheden en bestemmingen), structureert deze als associatielijst, en schrijft die naar bestand via call-with-output-file. Bij laad-scenario-in! gebeurt het omgekeerde: het bestand wordt veilig ingelezen via call-with-input-file, waarna de huidige treinen worden verwijderd en alle componenten iteratief teruggezet via NMBS-commando's.

Operaties:

NAAM	SIGNATUUR
<i>maak-scenario</i>	(nmbs $\rightarrow$ scenario)
<i>sla-scenario-op!</i>	(pad $\rightarrow \emptyset$)
<i>laad-scenario-in!</i>	(tekstbestand $\rightarrow \emptyset$)

3.12 Verduidelijking

In mijn project zijn er nog andere bestanden zoals server.rkt, client.rkt, gui.rkt, constanten.rkt en route.rkt. Deze bestanden zijn geen ADT's, omdat ze geen interne status encapsuleren via de dispatch-structuur die voor de andere componenten

gehanteerd wordt. Route.rkt biedt enkel een functie zoek-route aan voor het berekenen van het kortste pad via Breadth-First Traversal.

4. Afhankelijkheidsdiagram

Figuur 1 laat het afhankelijkheidsdiagram zien van de uitvoering van het project in fase 3. Een pijl tussen twee componenten wijst erop dat het ene onderdeel gebruikmaakt van de functies of informatie van het andere onderdeel; de pijl wijst dus naar de component waarvan men afhankelijk is. De voornaamste afhankelijkheden zijn:

- Client-zijde (GUI, Scenario, NMBS, Railway-NMBS): De GUI leest uit Railway-NMBS-cache en stuurt commando's via NMBS. NMBS communiceert via TCP met Infrabel en geeft inkomende statusberichten door aan Railway-NMBS. Scenario gebruikt NMBS om toestanden op te slaan en in te laden.
- Server-zijde (Infrabel + ADT's): Infrabel coördineert Trein, Wissel, Slagboom, Verkeerslicht, en is in fase 3 bovendien afhankelijk van Spoor netwerk, Reservatie en de Route-module. De Route-module gebruikt Spoor netwerk, dat zelf op de a-d-bibliotheek steunt. Hardware-abstractie: Infrabel praat enkel met de Execution Facade, die kiest tussen simulator en Z21-hardware. De GUI gebruikt daarnaast de Racket GUI-bibliotheek voor visualisatie.

Samengevat creëert deze structuur een strikte scheiding tussen de gebruikersinterface (Client) en de spoorweglogica (Server). De introductie in fase 3 van een graaf-gebaseerd Spoor netwerk, een Reservatiesysteem en een afzonderlijke Route-module maakt de architectuur niet alleen modulair en uitbreidbaar, maar ook volledig onafhankelijk van de fysieke of virtuele uitvoering van het spoor.

Figuur 1:

